International University

School of Computer Science and Engineering

Web Application Development

Laboratory

IT093IU

Lab #5 Practice

Submitted by

Nguyễn Hồng Ngọc Hân - ITCSIU22229

EXERCISE 1: PROJECT SETUP & MODEL LAYER

Task 1.1: Create Project Structure

Task 1.2: Create Student JavaBean

Explain: One student record is represented by the Student class, a JavaBean used in JSP/MVC applications. This class offers the public getters/setters, private fields, and no-argument constructor that a JavaBean needs. JSP pages and servlets can easily store, access, and display student data since each property (id, studentCode, fullName, email, major, createdAt) maps directly to a field in your database table. By printing the student's data in an understandable manner, the toString() method aids in troubleshooting.

Task 1.3: Create StudentDAO

Test Your DAO:

```
Student{id=21, studentCode='IT005', fullName='John Gem1', email='john.G@email.com', major='IT'}
Student{id=20, studentCode='IT003', fullName='John07', email='john07@email.com', major='IT'}
Student{id=18, studentCode='IT002', fullName='John06', email='john06@email.com', major='Computer Science'}
Student{id=17, studentCode='IT001', fullName='John05', email='John05@email.com', major='Computer Science'}
Student{id=16, studentCode='SV010', fullName='John03', email='', major='Computer Science'}
Student{id=14, studentCode='SV009', fullName='John', email='john@email', major='Computer Science'}
Student{id=11, studentCode='SV008', fullName='John Doe02', email='john.doe@company.co.uk        ', major='Computer Science'}
Student{id=10, studentCode='SV007', fullName='John Gem', email='john@email.com  ', major='Computer Science'}
Student{id=6, studentCode='SV006', fullName='John Doe', email='john.d@email.com', major='Computer Science       '}
Student{id=5, studentCode='SV005', fullName='David Wilson', email='david.w@email.com', major='Computer Science'}
Student{id=4, studentCode='SV004', fullName='Sarah Davis', email='sarah.d@email.com', major='Data Science'}
Student{id=3, studentCode='SV003', fullName='Michael Brown', email='michael.b@email.com', major='Software Engineering'}
Student{id=2, studentCode='SV002', fullName='Emily Johnson', email='emily.j@email.com', major='Information Technology'}
Student{id=1, studentCode='SV001', fullName='John Smith', email='john.smith@email.com', major='Computer Science'}
```

Explain:

The class StudentDAO is in charge of managing all student-related database operations. To load the MySQL driver and establish a connection to the database, it makes use of getConnection(). The getAllStudents() function selects all student records using an SQL query, uses setters to transform each row into a Student JavaBean, and delivers the results as a list so the JSP page can show the information.

EXERCISE 2: CONTROLLER LAYER

Task 2.1: Create Basic Servlet

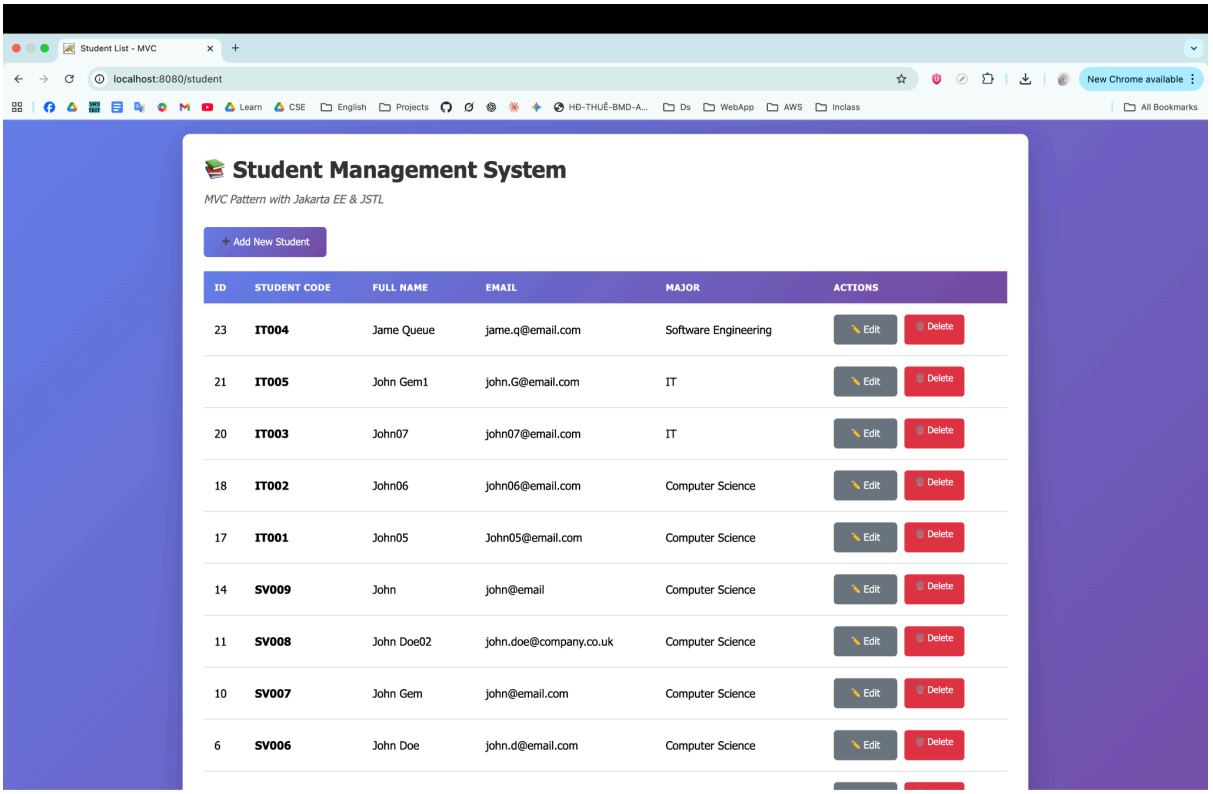
Task 2.2: Add More CRUD Methods

Explain:

- init() initializes the controller by creating a StudentDAO instance so all request-handling methods can access the database.
- doGet() acts as a router, deciding which action to execute based on the action parameter, such as listing students or showing a form.
- doPost() processes form data sent via POST and routes actions like inserting or updating a student.
- listStudents() gets all student records and forwards them to the JSP page to display the list.
- showNewForm() loads the student form with empty fields for adding a new record.
- showEditForm() loads a specific student's data into the form so the user can edit it.
- insertStudent() takes the submitted form data, creates a new student, and saves it to the database.
- updateStudent() updates a student's information in the database using the values submitted in the form.
- deleteStudent() removes a student record based on the given ID.

EXERCISE 3: VIEW LAYER WITH JSTL

Task 3.1: Create Student List View

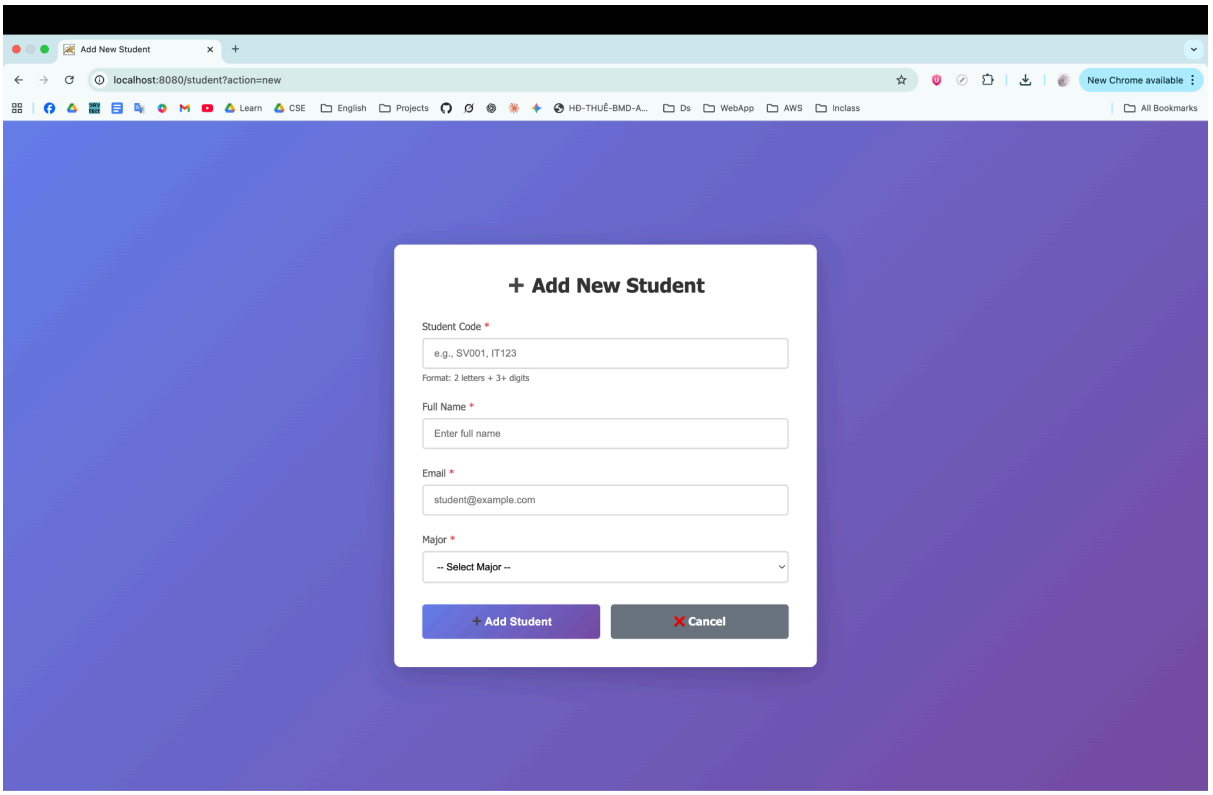


Explain:

This JSP file serves as the MVC pattern's view, which is in charge of presenting the student list in an appealing interface. The JSTL core tag library is included at the top, enabling the page to use JSTL tags for conditional rendering and looping through the student data, such as <c:if>, <c:forEach>, and <c:choose>. This view receives a list of students from the servlet controller, and JSTL is used to dynamically create the table rows, display success or error messages, and handle the case in which there are no students. The remaining portion of the file is made up of HTML and CSS that specify the Student Management page's overall user interface, buttons, and table design.

Task 3.2: Create Student Form View

*View after access the url /student?action=new



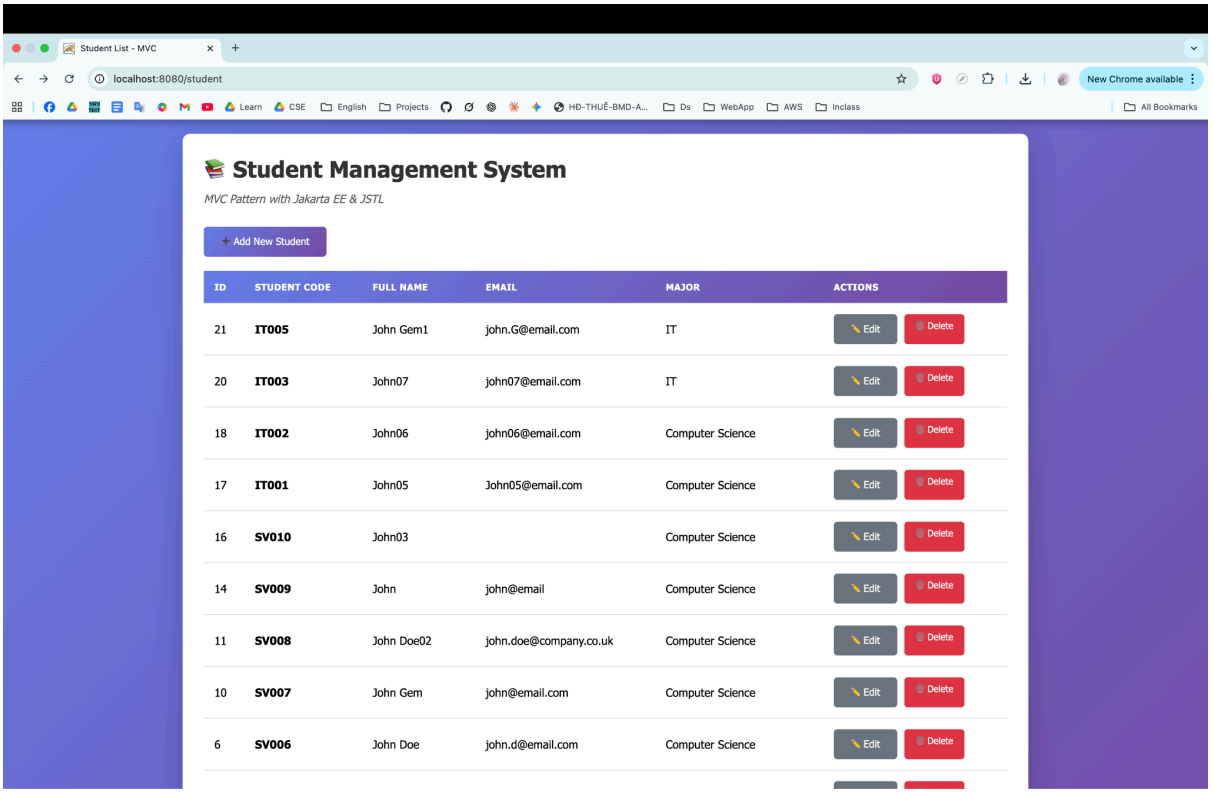
EXERCISE 4: COMPLETE CRUD OPERATIONS

Task 4.1: Complete DAO Methods

Task 4.2: Integration Testing

Test Sequence:

1. **List: Navigate to `/student`—should see existing students**

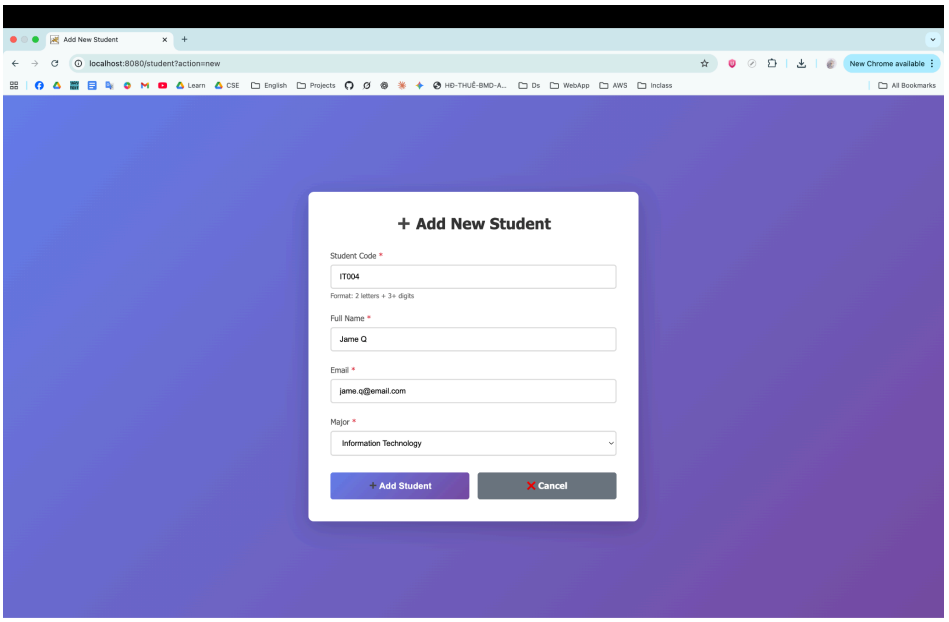


Explain:

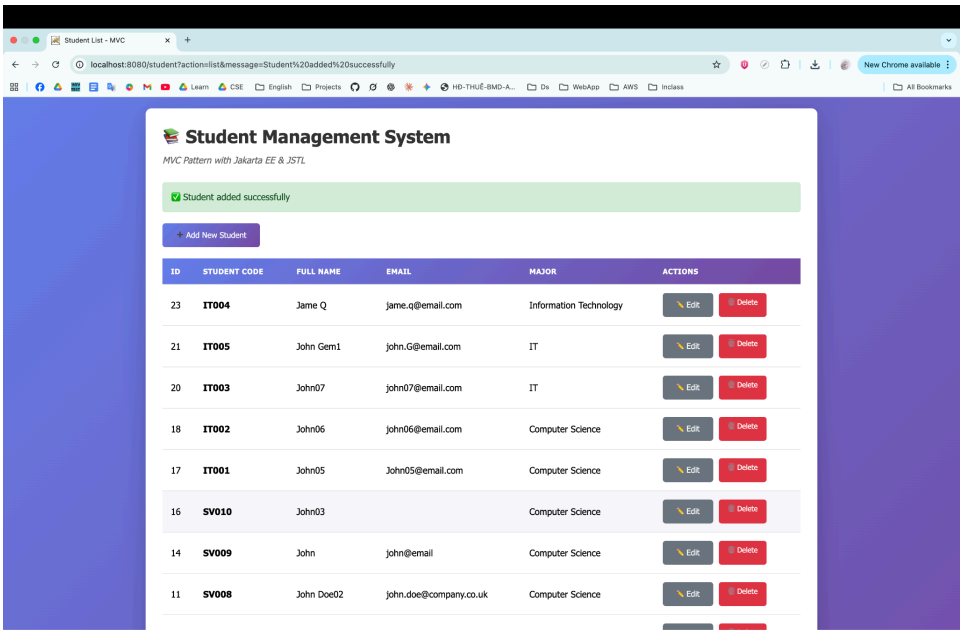
When the user accesses /students, the browser sends a GET request. Inside the controller, the doGet() method runs and handles the “list students” action following the standard MVC pattern. First, it calls the DAO to fetch all student records from the database. Once the data is retrieved, the controller stores the student list in the request object using setAttribute, allowing the JSP page to read it. Instead of creating a new response, the controller then forwards the request internally to student-list.jsp. Because this is a forward (not a redirect), the browser’s URL stays the same, but the JSP still receives the attached data and renders it on the page. In summary, the controller collects the data, adds it to the request, and forwards everything to the JSP view to display the list of students.

2. **Add: Click "Add New Student"**

- **Fill form with test data**



- **Submit**
- **Should redirect to list with success message**

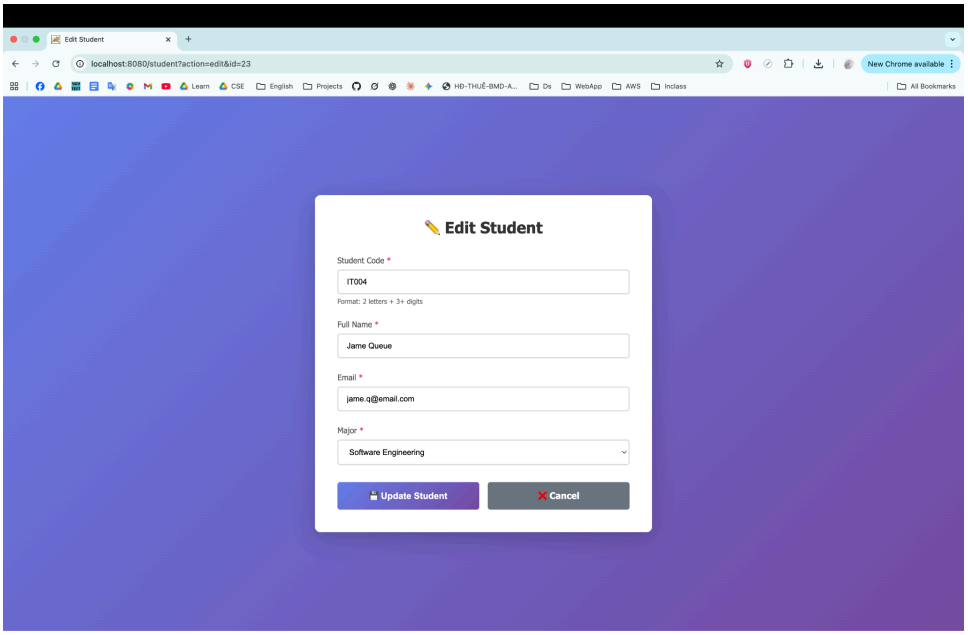


Explain:

When the user enters sample data into the form and submits it, the browser sends a POST request to the StudentController, which then calls the addStudent() method. Inside addStudent(), the system prepares an SQL INSERT statement using placeholders (“?”) to prevent SQL injection. It then opens a database connection and uses a PreparedStatement to safely assign each student value (student code,

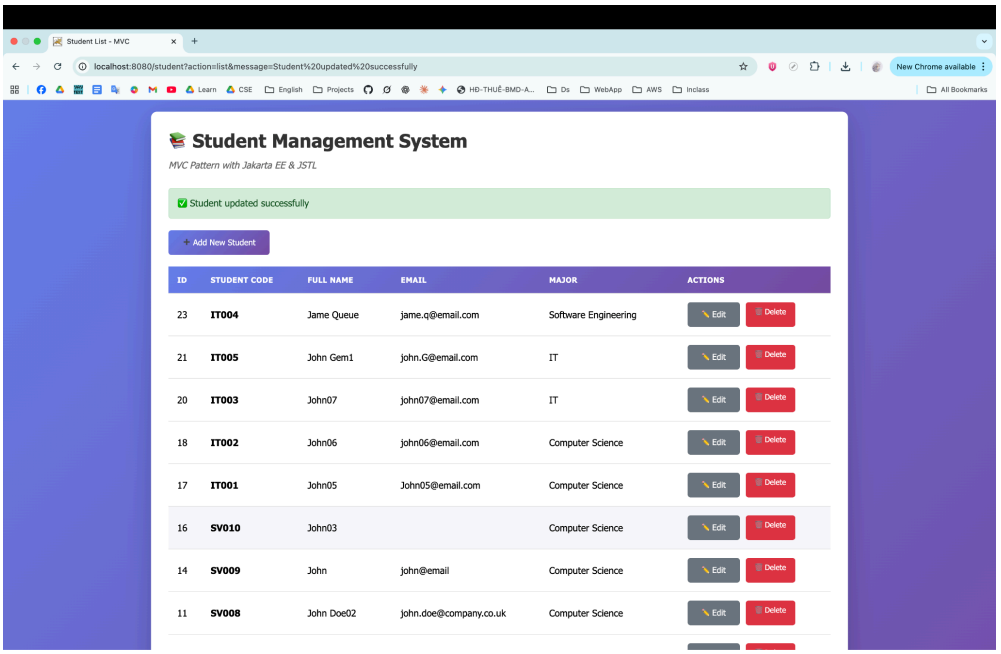
full name, email, major) to the correct placeholder. After all values are set, the method calls executeUpdate() to run the SQL command. This function returns the number of rows inserted into the database. If the result is greater than zero, it means the student was added successfully and the method returns true. If no rows are inserted or if any SQL error occurs, the method returns false.

3. **Edit: Click "Edit" on test student**
    - **Form should pre-fill**
    - **Modify data**



    - **Submit**

- **Should redirect with update message**



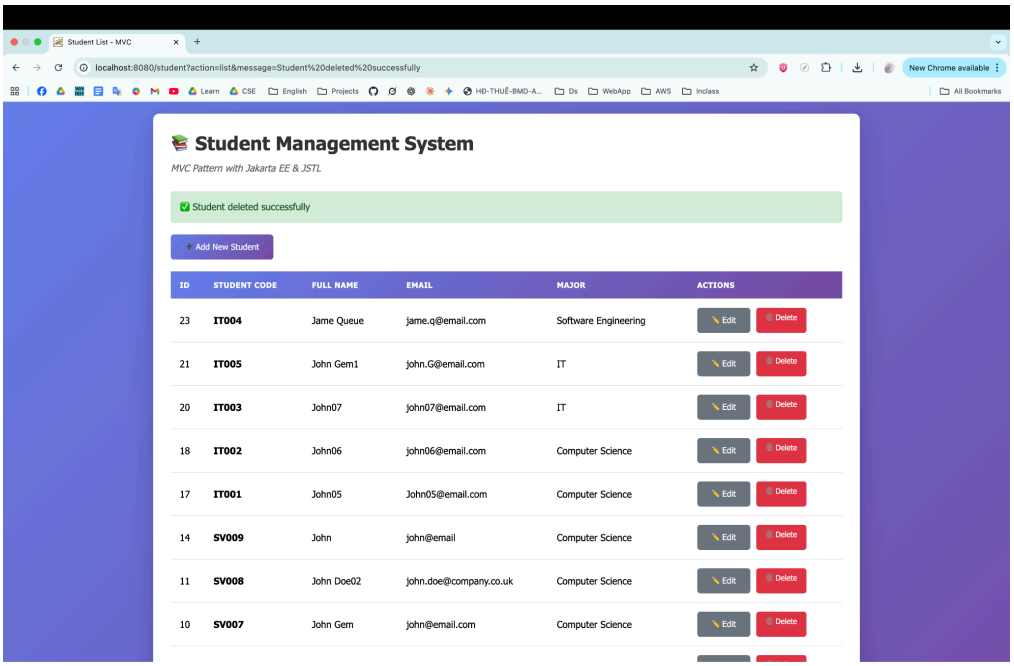
Explain:

The updateStudent() method is responsible for modifying an existing student's information in the database. It builds an SQL UPDATE statement that updates the student's code, name, email, and major based on the given id. The method opens a database connection, creates a PreparedStatement, and assigns each value from the Student object to the corresponding placeholder in the query.

After setting all parameters, it executes the statement using executeUpdate(), which returns the number of rows that were updated. If at least one row is successfully modified, the method returns true. If no rows are updated or if any error occurs, the method returns false.

4. **Delete: Click "Delete" on test student**
    - **Should confirm**

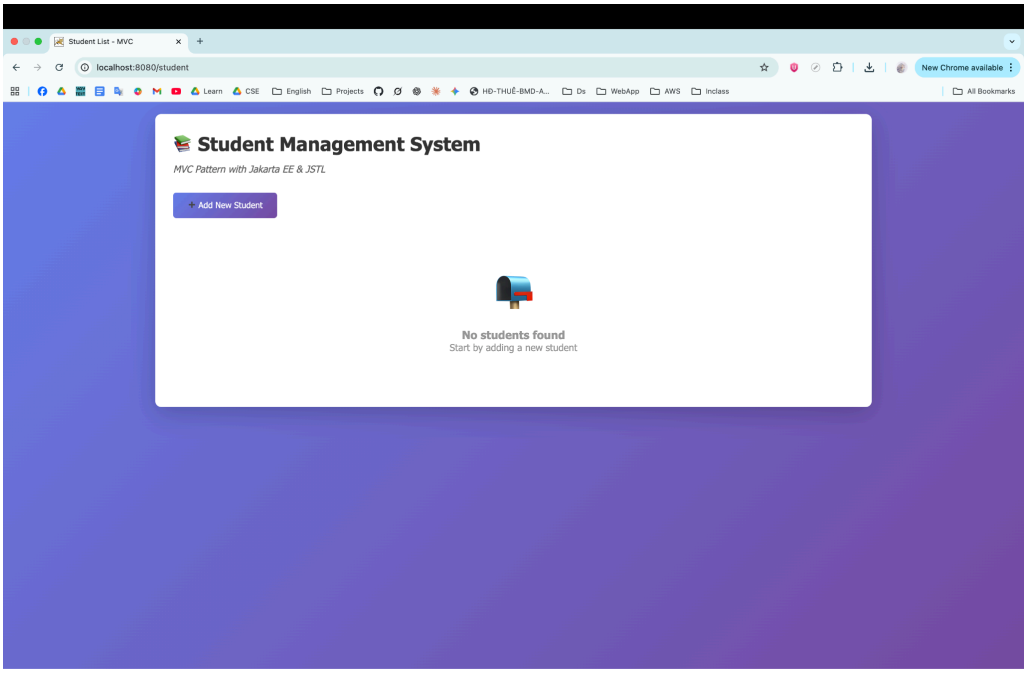
- **Should redirect with delete message**



Explain:

The deleteStudent() method deletes a student record from the database based on the provided ID. It creates an SQL DELETE statement containing a placeholder for the student ID, opens a database connection, and uses a PreparedStatement to safely assign the ID value. After that, it runs executeUpdate(), which returns the number of rows that were deleted. If at least one row is successfully removed, the method returns true. If no record matches the given ID or if any SQL error occurs, the method returns false.

5. **Empty State: Delete all students**
    - **Should show "No students found" message**



Explain:

The empty state is shown when the student list has no data—meaning there are no student records to display. Instead of rendering an empty table, the system displays a friendly message and an icon to let the user know that no students are available yet, along with a prompt to add a new one. This makes the interface more informative, user-friendly, and visually appealing.